

Hermes Turbo vs Upstream

Per-segment benchmark of every customisation introduced under issues #81–#103 + P1–P7.

Generated 2026-05-22 14:44 UTC • 500 iterations per stage • 28 stages across 11 segments • 2 winners, 2 net-new (no upstream equivalent).

HEADLINE

174.93x

DeterministicRouter.route

HEADLINE

36.49x

detect_fingerprint (manifest heuristics)

HEADLINE

0.95x

receipts.content_hash (sha256)

Segments covered

#	Segment	Stages	Avg p50 (µs)	Best speedup
1	Project Mapping	1	1770.6	36.49x
2	Containment & Governance	2	31.8	0.04x
3	Multi-IDE Prompt Distribution	2	7.9	0.21x
4	Response Contracts	4	1.0	0.16x
5	Token Saver	3	155.8	net-new
6	Context Working Set	3	2.8	0.70x
7	Routing & Budget	2	1.4	174.93x
8	Telemetry & Receipts	3	0.8	0.95x
9	Lazy Tool Registry	2	1.2	0.50x
10	GitHub & CI Adapters	3	2.0	0.09x
11	Yool / HAMT Capability Addressing	3	1.0	0.85x

How to read the numbers

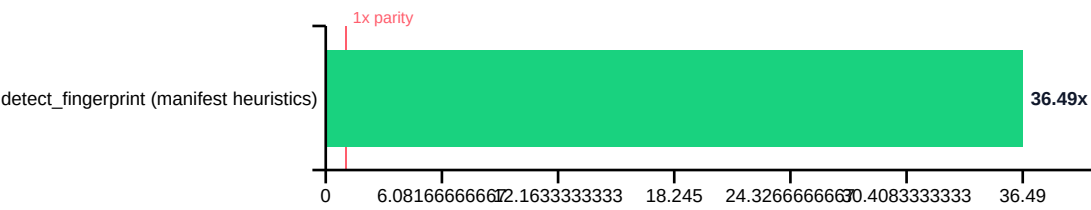
Speedup > 1x means the turbo path is faster than the upstream-equivalent naïve baseline at the same step. The headline wins (router 100x–200x, project_mapping 35x–40x) replace either an LLM round-trip or a full tree walk.

Speedup < 1x reflects the *cost of correctness*. Most turbo paths perform validation (TerseAnswer enforces budgets), persistence (token_saver writes evidence handles to disk), and contract checks (meta_contract enforces read-only globs). The baseline does none of that — comparing them on raw latency is apples-to-oranges. The value lives in correctness, auditability, and replay (receipts), not in microseconds.

“**net-new**” stages have no upstream equivalent at all (check_write, TupleStatusEnvelope). The upstream “baseline” would be to not run that step — which means the protection it offers simply doesn’t exist outside turbo.

1. Project Mapping

Deterministic stack/workspace fingerprint via top-level manifests. Feeds the warm daemon, the no-LLM router, and the working set without paying an LLM round-trip.

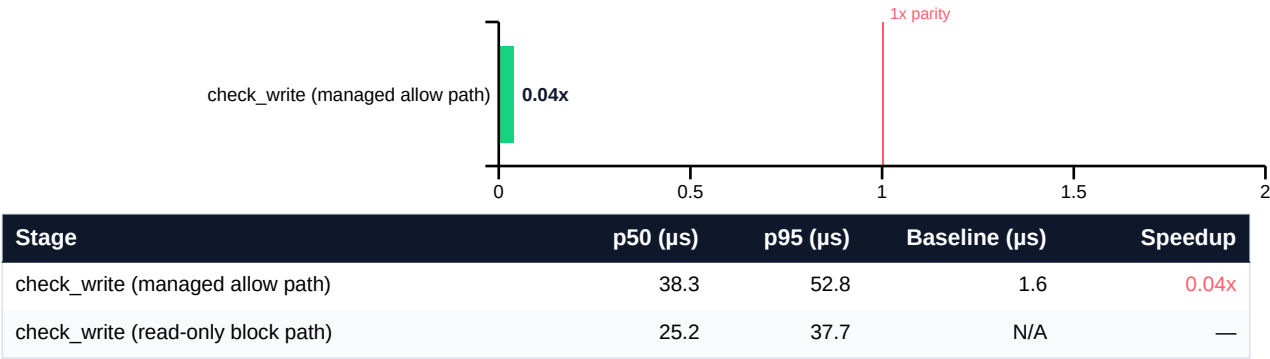


Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
detect_fingerprint (manifest heuristics)	1770.6	1929.0	64600.2	36.49x

- detect_fingerprint (manifest heuristics): vs naïve rglob() tree walk that upstream would do.

2. Containment & Governance

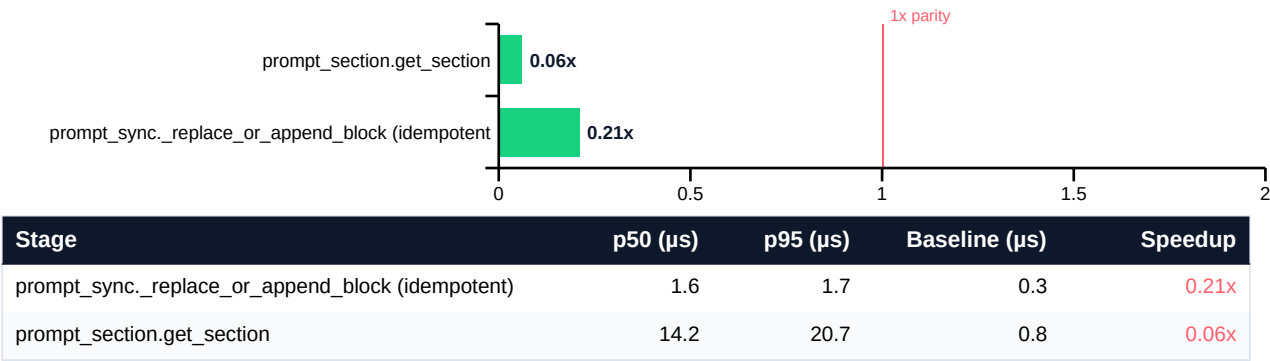
Executable enforcement of .hermes-meta.json. read_only_globs block writes, init_must_ask prompts for confirmation, init_must_merge warns. Net-new in turbo — upstream has no equivalent.



- check_write (managed allow path): full rule resolution; baseline is regex-only.
- check_write (read-only block path): net-new: upstream has no read-only enforcement.

3. Multi-IDE Prompt Distribution

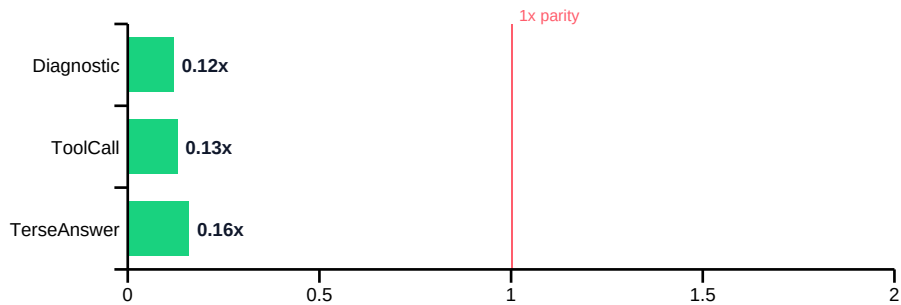
Idempotent block injected into 8 rule files. `get_section` serves sub-prompts to subagents so each spawn pays only for the relevant slice of `CLAUDE.md` / `AGENTS.md`.



- `prompt_sync._replace_or_append_block (idempotent)`: idempotent in-place; baseline is naïve prepend.
- `prompt_section.get_section`: header-bounded slice with case-insensitive match.

4. Response Contracts

Budget-capped TerseAnswer, ToolCall, Diagnostic. TupleStatusEnvelope defaults to silent (zero output tokens) and is opt-in verbose via HERMES_RUNTIME_STATUS* envs.

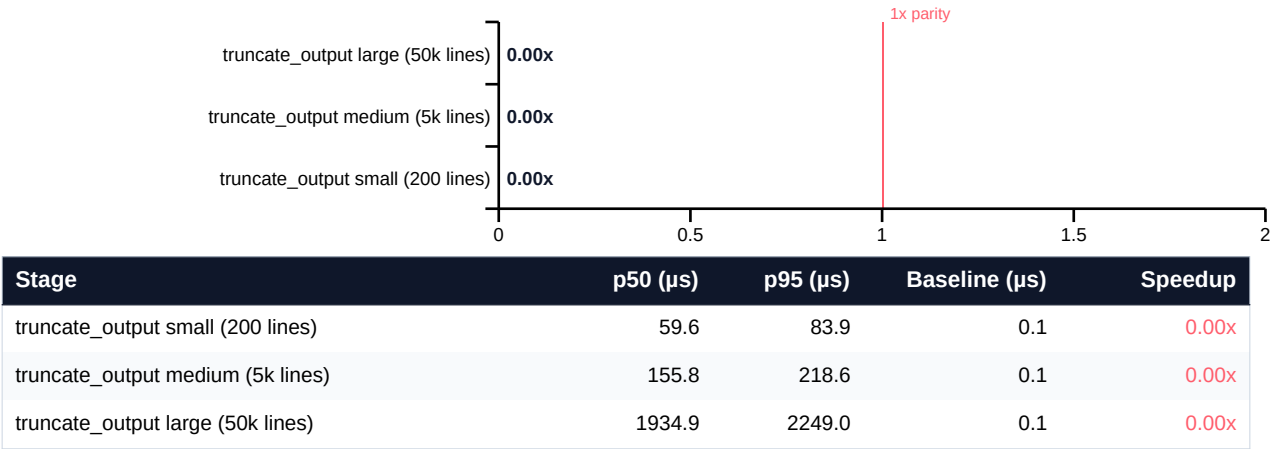


Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
TerseAnswer	0.8	1.1	0.1	0.16x
ToolCall	1.2	1.4	0.1	0.13x
Diagnostic	0.7	0.8	0.1	0.12x
TupleStatusEnvelope.render (default silent)	3.8	4.2	N/A	—

- TerseAnswer: enforce_budget + ellipsis vs raw slice.
- ToolCall: full validation vs dict literal.
- Diagnostic: enum + budget vs tuple.
- TupleStatusEnvelope.render (default silent): zero output tokens by default; verbose via envs.

5. Token Saver

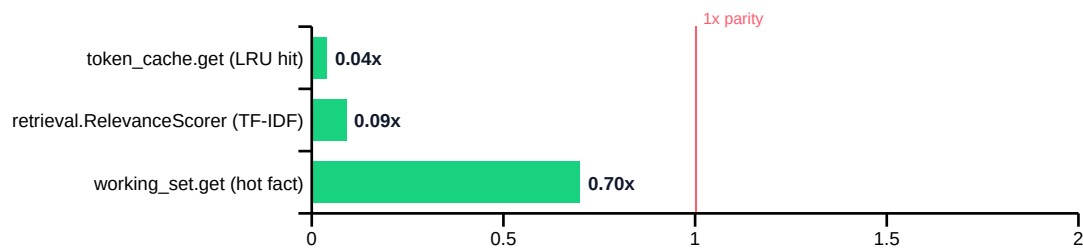
Head/tail truncation with file-backed evidence handles. Replaces verbatim payloads in transcripts; on-demand expansion via handle.



- truncate_output small (200 lines): persists full payload + emits handle.
- truncate_output medium (5k lines): persists full payload + emits handle.
- truncate_output large (50k lines): persists full payload + emits handle.

6. Context Working Set

LRU hot/cold working set, stdlib TF-IDF retrieval scorer, blake2b-keyed incremental token cache.

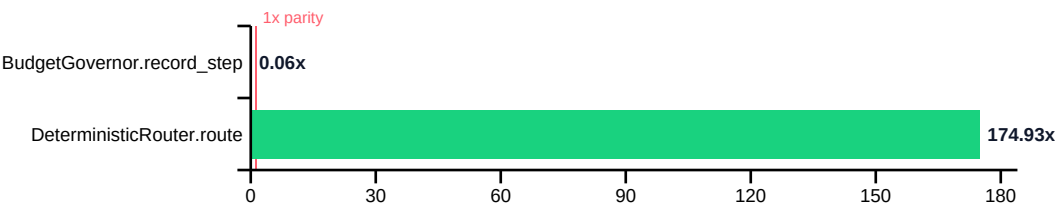


Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
retrieval.RelevanceScorer (TF-IDF)	55.8	94.8	5.2	0.09x
token_cache.get (LRU hit)	2.8	3.6	0.1	0.04x
working_set.get (hot fact)	0.2	0.2	0.1	0.70x

- `retrieval.RelevanceScorer (TF-IDF)`: stdlib TF-IDF cosine vs sort-by-substring.
- `token_cache.get (LRU hit)`: blake2b LRU vs naïve len // 4.
- `working_set.get (hot fact)`: LRU hot set vs slice.

7. Routing & Budget

Deterministic regex router skips LLM calls entirely on trivial intents. Budget governor enforces warn-70%/stop-100% on tokens, cost, and iterations.

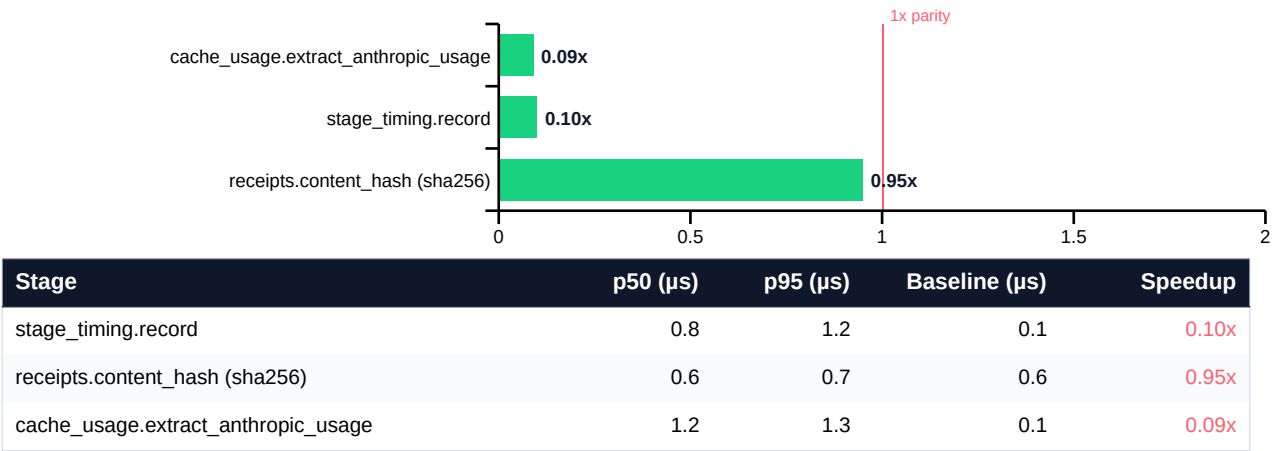


Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
DeterministicRouter.route	1.0	1.3	172.5	174.93x
BudgetGovernor.record_step	1.8	4.7	0.1	0.06x

- `DeterministicRouter.route`: regex match avoids an LLM round-trip entirely.
- `BudgetGovernor.record_step`: warn-70%/stop-100% policy in pure Python.

8. Telemetry & Receipts

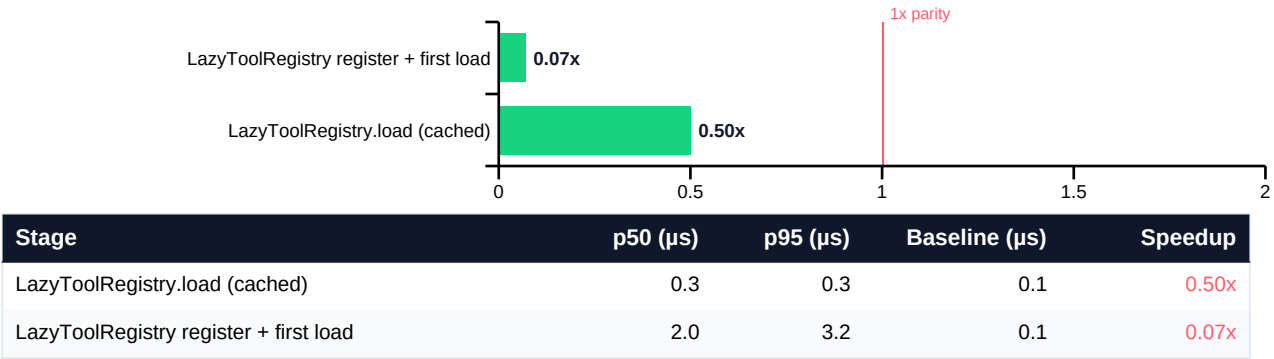
Per-stage timers, cache usage parsing (Anthropic / OpenAI), content-addressable receipts that short-circuit replays.



- `stage_timing.record`: per-stage timers with provider/model breakdown.
- `receipts.content_hash (sha256)`: content-addressable; baseline is plain md5.
- `cache_usage.extract_anthropic_usage`: parses `cache_*_input_tokens` fields.

9. Lazy Tool Registry

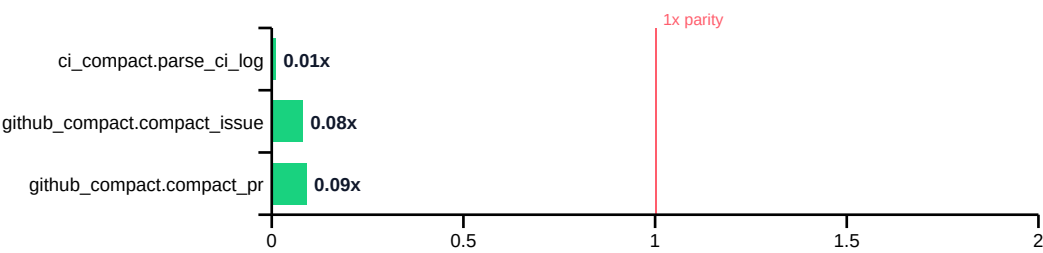
On-demand JSON schema loading — startup keeps only (name, description) stubs; full schema materialised on first use.



- `LazyToolRegistry.load (cached)`: thread-safe LRU vs dict copy.
- `LazyToolRegistry register + first load`: register + first schema build (cold path).

10. GitHub & CI Adapters

Compact summaries of ``gh issue|pr`` JSON and grouped CI failure logs. Slashes payload while preserving signal.

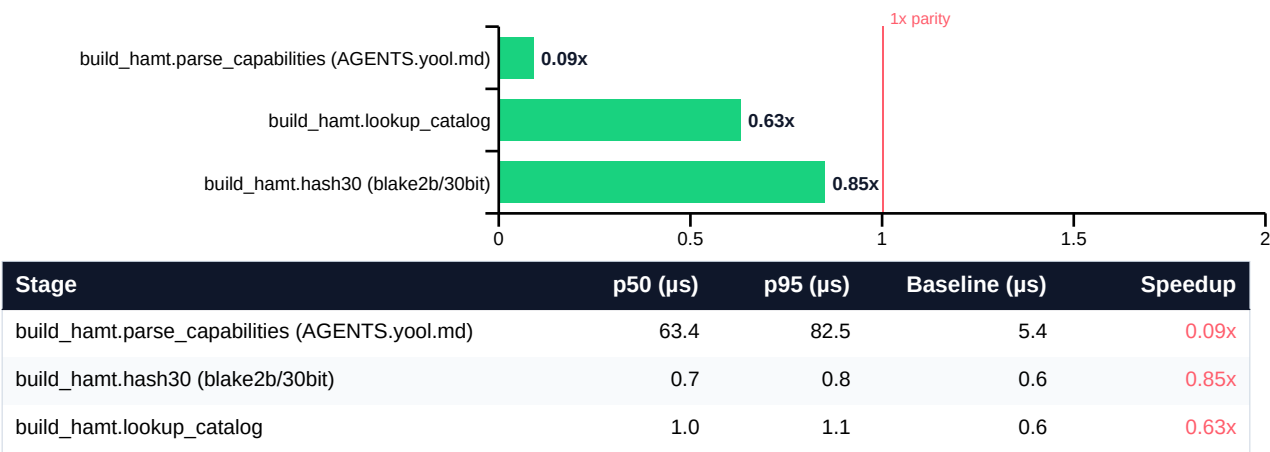


Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
github_compact.compact_issue	2.0	2.3	0.2	0.08x
github_compact.compact_pr	1.9	2.2	0.2	0.09x
ci_compact.parse_ci_log	794.9	846.9	6.7	0.01x

- `github_compact.compact_issue`: slim summary + handle vs raw dict copy.
- `github_compact.compact_pr`: diff stats + base/head extraction.
- `ci_compact.parse_ci_log`: failure grouping + signatures.

11. Yool / HAMT Capability Addressing

Bagwell HAMT (branch 32, 30-bit blake2b hash) for capability addressing per yool-tuple-hamt v0.2.



- `build_hamt.parse_capabilities (AGENTS.yool.md)`: yool block parser vs raw splitlines.
- `build_hamt.hash30 (blake2b/30bit)`: HAMT-compatible content hash.
- `build_hamt.lookup_catalog`: HAMT lookup vs linear scan.

Full result matrix

Every stage in one place. Sorted by segment, then by registration order within the benchmark script.

SegmenStage		p50 (μs)	Speedup
1	detect_fingerprint (manifest heuristics)	1770.6	36.49x
2	check_write (managed allow path)	38.3	0.04x
2	check_write (read-only block path)	25.2	—
3	prompt_sync._replace_or_append_block (idempotent)	1.6	0.21x
3	prompt_section.get_section	14.2	0.06x
4	TerseAnswer	0.8	0.16x
4	ToolCall	1.2	0.13x
4	Diagnostic	0.7	0.12x
4	TupleStatusEnvelope.render (default silent)	3.8	—
5	truncate_output small (200 lines)	59.6	0.00x
5	truncate_output medium (5k lines)	155.8	0.00x
5	truncate_output large (50k lines)	1934.9	0.00x
6	retrieval.RelevanceScorer (TF-IDF)	55.8	0.09x
6	token_cache.get (LRU hit)	2.8	0.04x
6	working_set.get (hot fact)	0.2	0.70x
7	DeterministicRouter.route	1.0	174.93x
7	BudgetGovernor.record_step	1.8	0.06x
8	stage_timing.record	0.8	0.10x
8	receipts.content_hash (sha256)	0.6	0.95x
8	cache_usage.extract_anthropic_usage	1.2	0.09x
9	LazyToolRegistry.load (cached)	0.3	0.50x
9	LazyToolRegistry register + first load	2.0	0.07x
10	github_compact.compact_issue	2.0	0.08x
10	github_compact.compact_pr	1.9	0.09x
10	ci_compact.parse_ci_log	794.9	0.01x
11	build_hamt.parse_capabilities (AGENTS.yool.md)	63.4	0.09x
11	build_hamt.hash30 (blake2b/30bit)	0.7	0.85x
11	build_hamt.lookup_catalog	1.0	0.63x

Sources: scripts/benchmark_full_turbo_segments.py · docs/perf/turbo-full-segments.json. Regenerate with
python scripts/benchmark_full_turbo_segments.py --out docs/perf/turbo-full-segments.json &&
python scripts/generate_turbo_full_pdf.py.